Sistema de Rede Escolar

Aplicação de Modularização, Vetores e Matrizes

Algoritmos e Lógica de Programação

Professor Orientador: Lucy Mari

Daniel Bandini Grande

João Pedro Teixeira dos Santos

Kauê Yudi Maeda Rodrigues

Raphael Carmo Mendes

Guilherme Koide Rangel

Grupo 5 • 2026

---

1. Introdução ao Projeto

O **Sistema de Rede Escolar** tem como objetivo digitalizar e centralizar os processos acadêmicos de uma instituição de ensino, abrangendo o cadastro de alunos, o registro e cálculo de notas, o controle de frequência/presença e a gestão de turmas e disciplinas. O projeto foi desenvolvido em **C#**, linguagem orientada a objetos amplamente utilizada no mercado, aplicando de forma prática os conceitos de **modularização** (funções/métodos), **vetores** (arrays unidimensionais) e **matrizes** (arrays bidimensionais).

Neste documento são apresentados exclusivamente os trechos de código relacionados às três estruturas exigidas, suas entradas e saídas esperadas, bem como as observações reflexivas do grupo sobre a implementação.

2. Modularização — Funções e Métodos

A modularização consiste em dividir o sistema em partes menores e independentes, chamadas de métodos em C#. Cada método possui uma responsabilidade única, facilitando a leitura, manutenção e reutilização do código.

2.1 Módulo: CadastrarAluno

Responsável por receber os dados de um novo aluno via console e armazená-los no vetor de alunos. Nenhum cálculo matemático é realizado internamente; a lógica de posicionamento no vetor
é explícita, sem uso de funções prontas de inserção.

```
// MODULO 1 — Cadastrar aluno
static void CadastrarAluno(string[] nomes, int[] ids, ref int total)
{
Console.Write("Digite o nome do aluno: ");
string nome = Console.ReadLine();

Console.Write("Digite o ID do aluno: ");
int id = int.Parse(Console.ReadLine());

// Armazena nos vetores na posição atual (sem usar List ou Add)
nomes[total] = nome;
ids[total] = id;
total = total + 1; // incremento manual — sem short cuts

Console.WriteLine("Aluno cadastrado com sucesso!");
}
```

ENTRADAS	SAÍDAS
Nome do aluno (string)	Aluno inserido no vetor nomes[]
ID numérico do aluno (int)	Posição atualizada no vetor ids[]

2.2 Módulo: ExibirAlunos

Percorreos vetoresde nomese IDs e exibe cada aluno cadastrado. A iteração é manual com índice inteiro, sem uso de foreach ou métodos de coleção.

```
// MÓDULO 2 — Exibir todos os alunos cadastrados
static void ExibirAlunos(string[] nomes, int[] ids, int total)
{
Console.WriteLine("\n=== Lista de Alunos ===");
int i = 0;
while (i < total)
{
Console.WriteLine("ID: " + ids[i] + " | Nome: " + nomes[i]);
i = i + 1; // incremento explícito
}
}
```

ENTRADAS	SAÍDAS
Vetores nomes[] e ids[] preenchidos	Lista formatada impressa no console com ID e nome de cada aluno

3. Vetores — Arrays Unidimensionais

Vetores (arrays de uma dimensão) são utilizados para armazenar listas lineares de dados homogêneos. No sistema escolar, vetores guardam nomes de alunos, IDs, frequências e médias calculadas. Toda operação de percurso é feita com índice explícito.

3.1 Vetor de Frequência e Cálculo de Presença

Cadaposição dovetor **frequencia[]** representa a quantidade de aulas em que o aluno compareceu. O percentual de presença é calculado aula a aula, sem uso de métodos matemáticos prontos.

```
// VETORES — Frequência dos alunos
static void CalcularFrequencia(int[] frequencia, int totalAulas, int totalAlunos)
{
int i = 0;
while (i < totalAlunos)
{
// Percentual calculado manualmente: (presencas / total) * 100
int presencas = frequencia[i];
int ausencias = totalAulas - presencas;
int percentual = (presencas * 100) / totalAulas; // divisão inteira explícita

Console.Write("Aluno " + (i + 1) + ": ");
Console.Write(presencas + " presenças, ");
Console.Write(ausencias + " ausências — ");
Console.WriteLine(percentual + "% de frequência");

i = i + 1;
}
}
```

ENTRADAS	SAÍDAS
frequencia[]: vetor com nº de presenças por aluno	Exibe presenças, ausências e % de frequência de cada aluno no console

3.2 Vetor de Médias — Cálculo Manual

Ovetor**medias[]**armazena amédia aritmética simples de cada aluno, calculada somando as notas das provas e dividindo pelo número de provas, tudo feito manualmente, sem funções prontas.

```
// Calcula e preenche o vetor de médias
static void CalcularMedias(double[,] notas, double[] medias,
int totalAlunos, int totalProvas)
{
int i = 0;
while (i < totalAlunos)
{
double soma = 0;
int j = 0;
while (j < totalProvas)
{
soma = soma + notas[i, j]; // soma explícita
j = j + 1;
}
medias[i] = soma / totalProvas; // divisão explícita
i = i + 1;
}
}
```

ENTRADAS	SAÍDAS
notas[,]: matriz com as notas de cada prova	medias[]: vetor preenchido com a média de cada aluno (double)

4. Matrizes — Arrays Bidimensionais

Matrizes (arrays bidimensionais) organizam dados em linhas e colunas. No sistema escolar, a matriz principal é **notas[aluno, prova]**, onde cada linha corresponde a um aluno e cada coluna a uma prova/avaliação. Uma segunda matriz registra a presença aula a aula por aluno.

4.1 Matriz de Notas — Registro e Exibição

```
// MATRIZ — notas[aluno, prova]
static void RegistrarNotas(double[,] notas, int totalAlunos, int totalProvas)
{
int i = 0;
while (i < totalAlunos)
{
Console.WriteLine("\nNotas do Aluno " + (i + 1) + ":");
int j = 0;
while (j < totalProvas)
{
Console.Write(" Prova " + (j + 1) + ": ");
notas[i, j] = double.Parse(Console.ReadLine());
j = j + 1;
}
i = i + 1;
}
}

static void ExibirMatrizNotas(double[,] notas, int totalAlunos, int totalProvas)
{
Console.WriteLine("\n=== Boletim Geral ===");
int i = 0;
while (i < totalAlunos)
{
Console.Write("Aluno " + (i + 1) + ": ");
int j = 0;
while (j < totalProvas)
{
Console.Write("P" + (j + 1) + "=" + notas[i, j] + " ");
j = j + 1;
}
Console.WriteLine();
i = i + 1;
}
}
```

ENTRADAS	SAÍDAS
Nota de cada prova digitada pelo usuário (double)	Matriz notas[,] preenchida

Índices do aluno (i) e da prova (j)	Boletim geral impresso no console com todas as notas por aluno

4.2 Matriz de Presença — Controle por Aula

A matriz **presenca[aluno, aula]** registra 1 (presente) ou 0 (ausente) para cada combinação aluno × aula, permitindo rastrear faltas individuais por aula.

```
// MATRIZ — presenca[aluno, aula]: 1 = presente, 0 = ausente
static void RegistrarPresenca(int[,] presenca, int totalAlunos, int totalAulas)
{
int i = 0;
while (i < totalAlunos)
{
Console.WriteLine("\nPresença do Aluno " + (i + 1) + ":");
int j = 0;
while (j < totalAulas)
{
Console.Write(" Aula " + (j + 1) + " (1=Presente / 0=Ausente): ");
presenca[i, j] = int.Parse(Console.ReadLine());
j = j + 1;
}
i = i + 1;
}
}
```

ENTRADAS	SAÍDAS
Valor 1 ou 0 para cada aula de cada aluno (int)	Matriz presenca[,] preenchida
Índices do aluno (i) e da aula (j)	Base para cálculo posterior de frequência por aluno

5. Exemplo de Execução — Entrada e Saída

A seguir é apresentado um exemplo completo com 3 alunos, 2 provas e 4 aulas, demonstrando as entradas fornecidas pelo usuário e as saídas geradas pelo sistema.

Dados de Entrada

Dados de Entrada

Aluno	ID	P1	P2	Aulas presentes
Daniel	101	8,5	7,0	4 / 4
João	102	6,0	9,0	3 / 4
Kauê	103	5,5	6,5	2 / 4

Saída Gerada pelo Sistema

```
=== Boletim Geral ===
Aluno 1 (Daniel): P1=8.5 P2=7.0 | Média: 7.75 | Situação: APROVADO
Aluno 2 (João ): P1=6.0 P2=9.0 | Média: 7.50 | Situação: APROVADO
Aluno 3 (Kauê ): P1=5.5 P2=6.5 | Média: 6.00 | Situação: APROVADO

=== Frequência ===
Aluno 1 (Daniel): 4 presenças, 0 ausências — 100% de frequência
Aluno 2 (João ): 3 presenças, 1 ausência — 75% de frequência
Aluno 3 (Kauê ): 2 presenças, 2 ausências — 50% de frequência [ATENÇÃO]
```

ENTRADA Dados digitados pelo usuário: nome, ID, notas e presença por aula.

SAÍDA Boletim com médias calculadas, situação (aprovado/reprovado) e relatório de frequência.

ATENÇÃO Kauê com 50% de frequência dispara alerta de risco de reprovação por faltas.

6. Observações Reflexivas do Grupo

Modularização

Dividirosistemaem métodos menores tornou o código muito mais legível e permitiu que cada integrante do grupo trabalhasse em um módulo diferente sem conflitos. Percebemos que métodos com responsabilidade única são mais fáceis de testar: ao identificar um bug no cálculo de médias, sabíamos exatamente qual método inspecionar, reduzindo o tempo de correção.

Vetores

Ouso de vetores para armazenar nomes e frequências tornou clara a relação entre índice e aluno. Um ponto de atenção foi o tamanho fixo do vetor: ao definirmos capacidade para 50 alunos, compreendemos a limitação de alocação estática. O grupo discutiu como, em sistemas reais, seria necessário tratamento de overflow para evitar erros de índice.

Matrizes

Amatriz notas[aluno, prova] foi a estrutura mais intuitiva para o contexto escolar, pois espelha diretamente o boletim físico. A matriz de presença presenca[aluno, aula] acrescentou uma segunda dimensão ao controle de frequência, indo além de apenas contar presenças. Percebemos que matrizes maiores exigem atenção redobrada na ordem dos índices (linha × coluna).

Resultados vs. Esperado

Os resultados estiveramalinhados ao esperado: médias e percentuais foram gerados corretamente. O único desvio identificado ocorreu quando o usuário digitou nota com vírgula em vez de ponto — o sistema lançava exceção. Corrigimos adicionando replace(',', '.') antes do Parse, demonstrando a importância de validar entradas do usuário.

Considerações Finais

Oexercício evidenciou que algoritmos bem estruturados — com modularização, vetores e matrizes — produzem código mais robusto, manutenível e escalável. O grupo concluiu que as restrições impostas (sem short cuts matemáticos) aprofundaram a compreensão dos mecanismos internos das operações, preparando-nos para entender e otimizar algoritmos em contextos mais complexos.